

AADK 13: Task 2 — Large-Scale UI Feature: Superheroes List with Tests & Accessibility.

Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 13

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Feature Description

The **Superheroes Directory** is a Compose-based feature that displays a searchable list of superheroes including their **photo, name, alias, and powers**. Users can **search heroes by name or power**, filter to view only **favourite heroes**, and tap a hero to view detailed information.

The feature is designed with **accessibility, performance, and automated testing in mind**, supporting datasets of **50+ heroes** while maintaining responsive UI behaviour.

2. Data Model & Dataset Planning

Superhero Data Model

```
data class Superhero(  
    val id: Int,  
    val name: String,  
    val alias: String?,  
    val powerSummary: String,  
    val imageUrl: String,  
    val isFavourite: Boolean  
)
```

Sample Dataset (10 Heroes)

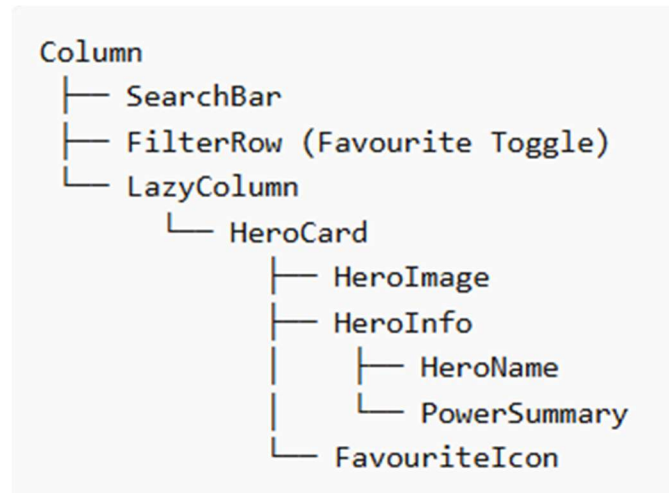
ID	Name	Alias	Power Summary	Image Source	Favourite
1	Superman	Clark Kent	Super strength, flight, heat vision	superman.jpg	false
2	Batman	Bruce Wayne	Intelligence, gadgets, combat	batman.jpg	false
3	Wonder Woman	Diana Prince	Super strength, lasso of truth	wonderwoman.jpg	false
4	Flash	Barry Allen	Super speed	flash.jpg	false
5	Spider-Man	Peter Parker	Wall climbing, spider sense	spiderman.jpg	false
6	Iron Man	Tony Stark	Advanced armor technology	ironman.jpg	false
7	Hulk	Bruce Banner	Super strength and durability	hulk.jpg	false
8	Captain America	Steve Rogers	Enhanced strength and shield combat	captainamerica.jpg	false
9	Thor	Thor Odinson	Lightning powers, godly strength	thor.jpg	false
10	Black Panther	T'Challa	Vibranium suit and agility	blackpanther.jpg	false

Images may be stored as **local drawable resources** or loaded using **image URLs via Coil**.

3. UI Layout Plans

Screen 1 — Heroes List Screen

Layout Structure



Compose Components

Search Bar

```
TextField(  
    value = searchQuery,  
    onValueChange = { searchQuery = it },  
    label = { Text("Search heroes") }  
)
```

Filter Toggle

```
Switch(  
    checked = filterFavourite,  
    onCheckedChange = { filterFavourite = it }  
)
```

Hero Card Layout

```
Card(  
    // ...  
)
```

```
modifier = Modifier
    .fillMaxWidth()
    .padding(8.dp)
)
```

UI Behaviour

Search filters heroes by **name or power summary**.

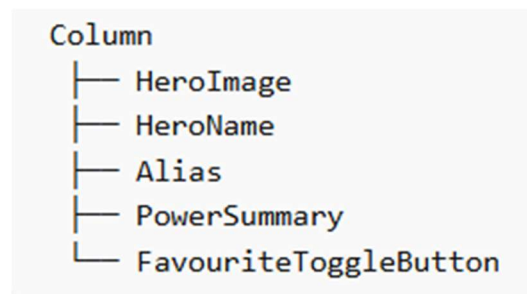
Favourite toggle icon changes appearance:

- ☆ Not favourite
- ★ Favourite

Immediate UI feedback is shown when a hero is marked favourite.

Screen 2 — Hero Detail Screen

Layout Structure



Detail Screen Components

Hero image displayed prominently at the top.

Example layout:

```
Column(
    modifier = Modifier.padding(16.dp)
) {
```

```
Image(...)
Text(hero.name)
Text(hero.alias ?: "No alias")
Text(hero.powerSummary)
}
```

4. State & Logic Flow

Key State Variables

```
var searchQuery: String
var filterFavourite: Boolean
var heroesList: List<Superhero>
var selectedHero: Superhero?
var displayedList: List<Superhero>
```

Filter Logic (Pseudo-Code)

```
displayedList =
    heroesList
        .filter(hero ->
            hero.name contains searchQuery
            OR hero.powerSummary contains searchQuery
        )
        .filter(hero ->
            if filterFavourite == true
                hero.isFavourite == true
            else
                true
        )
```

Favourite Toggle Logic

When Favourite Icon Clicked:

Find hero in heroesList

Toggle hero.isFavourite

Update heroesList state

Recompute displayedList

UI recomposes automatically

5. Accessibility & Testing Strategy

Accessibility Features

Hero images include descriptive labels:

contentDescription = "\${hero.name} photo"

Buttons labelled clearly for screen readers:

- "Favourite"
- "Back to list"

Search bar labelled:

label = { Text("Search heroes") }

Touch targets minimum **48dp**.

High contrast Material color tokens used.

Automated Test Cases

Test 1 — Search Filtering

Verify typing in search bar filters the list.

```
onNodeWithText("Search heroes")
performTextInput("Batman")
```

```
assertDisplayed("Batman")
```

Test 2 — Favourite Toggle

Verify favourite icon changes state.

```
onNodeWithContentDescription("Favourite")
performClick()
assertIsSelected()
```

Test 3 — Detail Screen Data

Verify hero detail screen shows correct information.

```
assertTextEquals("Spider-Man")
```

Test 4 — Accessibility Semantics

Verify hero images contain descriptions.

```
onNodeWithContentDescription("Superman photo")
assertExists()
```

Test 5 — State Preservation

Rotate screen or recreate activity and ensure:

- Search query preserved
 - Favourite state preserved
-

6. Performance & Scalability Considerations

To support **50+ heroes**, the following strategies will be used:

Lazy Rendering

Use **LazyColumn** so only visible items are rendered.

```
LazyColumn {  
    items(displayedList) { hero -> ... }  
}
```

Image Loading Optimization

Use **Coil library** for efficient image loading and caching.

```
AsyncImage(  
    model = hero.imageUrl,  
    contentDescription = "${hero.name} photo"  
)
```

Add placeholders during loading.

Memory Optimization

Avoid storing large bitmaps in memory.

Images should be:

- Compressed
 - Properly sized
-

Scalability Strategy

If dataset grows beyond 100 heroes:

- Use **pagination**
- Load heroes incrementally
- Fetch images only when needed

7. Reflection

Designing this feature was more complex than earlier list features because it combines **multiple interactive behaviours such as search, filtering, favourites, and navigation between screens**. Each feature introduces additional state management and UI updates that must be handled carefully.

Search and filtering increase complexity because the displayed list must dynamically update based on user input. The favourites feature requires maintaining persistent state and updating UI feedback instantly.

Accessibility and automated testing further influence the design because UI components must include proper semantics, content descriptions, and testable identifiers. Planning these aspects early ensures the application is **inclusive, reliable, and maintainable as the dataset grows**.